



Manipulación de datos y automatización con R

Introducción a la programación con R

Mg. J. Eduardo Gamboa U.

2026-07-22

Estructuras de datos

Una estructura de datos es una forma organizada de almacenar y agrupar información dentro de un programa para facilitar su acceso, modificación y procesamiento.

Vector

- ▶ Un vector es una estructura de datos unidimensional que almacena varios elementos del mismo tipo.
- ▶ Cuando se combinan elementos de diferentes tipos, R convierte todos los elementos al tipo más general.
- ▶ Se accede a sus elementos mediante un índice.

Lista

- ▶ Una lista es una estructura que puede almacenar elementos de diferentes tipos, tamaños y estructuras.
- ▶ Cada componente de una lista puede ser un vector, una matriz, un data frame, otra lista o cualquier otro objeto de R.
- ▶ Sus elementos pueden tener diferentes longitudes.
- ▶ Sus elementos pueden ser nombrados.
- ▶ Son útiles para devolver varios resultados desde una función.

Matriz

- ▶ Una matriz es una estructura bidimensional formada por filas y columnas.
- ▶ Todos sus elementos deben ser del mismo tipo de dato.
- ▶ Son apropiadas para cálculos algebraicos y numéricos.
- ▶ Se accede a sus elementos mediante dos índices.

Estructuras de control

Las estructuras de control permiten modificar el flujo normal de ejecución de un programa. Se utilizan para tomar decisiones y ejecutar determinadas instrucciones según se cumpla o no una condición.

Estructura if

La estructura if ejecuta un bloque de código únicamente cuando una condición es verdadera, la cual debe producir un único valor lógico: TRUE o FALSE.

Ejemplo 1

Cree un objeto llamado edad con el valor 20. Utilice una estructura if para mostrar el mensaje “Mayor de edad” ya que la edad es mayor o igual a 18.

Ejemplo 2

Considere el siguiente data frame con los códigos de cinco estudiantes y sus respectivas notas:

```
notas <- data.frame(Alumno = c("001", "002", "008", "015", "025"),  
                    Nota = c(12, 10, 15, 18, 19))
```

Utilice una estructura if para verificar si todas las notas son mayores que 10. Si la condición se cumple, muestre el mensaje “Todas las notas son aprobatorias”.

Ejemplo 3

Considere el vector:

```
x <- c(4, 7, 9, 12)
```

Utilice una estructura if para verificar que x sea numérico. Si no lo es, detenga la ejecución con el mensaje “El argumento x debe ser numérico”. Si la validación es correcta, calcule la media del vector.

Ejemplo 4

Considere el objeto:

```
x <- letters
```

Utilice una estructura if para verificar si x es numérico. En caso contrario, detenga la ejecución mediante stop() y muestre el mensaje “El argumento x debe ser numérico”.

Estructura if...else

La estructura if...else permite elegir entre dos alternativas, donde solo uno de los dos bloques será ejecutado.

Ejemplo 5

Cree un objeto llamado edad con el valor 16. Utilice una estructura if...else para mostrar el mensaje “Mayor de edad” cuando la edad sea mayor o igual a 18; en caso contrario, muestre “Menor de edad”. (En este caso debe mostrar Menor de edad).

Ejemplo 6

Considere un data frame que contiene los códigos de cinco estudiantes y sus respectivas notas. Utilice una estructura if...else para verificar si existe al menos una nota mayor que 10. Si la condición se cumple, muestre el mensaje “Existe al menos una nota mayor que 10”; de lo contrario, muestre “Ningún estudiante obtuvo una nota mayor que 10”.

Ejemplo 7

Considere el vector `m <- c(3, 5, 7, 9, 11)`. Utilice una estructura `if...else` para determinar si el vector contiene una cantidad par o impar de elementos. Muestre el mensaje correspondiente para cada caso. (En el vector indicado debe mostrar que la cantidad es impar).

Ejemplo 8

Considere una matriz construida con los valores 12, 8, 15, 10, 17 y 14, organizada en dos filas. Utilice una estructura `if...else` para determinar si la matriz es cuadrada. Si el número de filas es igual al número de columnas, muestre “La matriz es cuadrada”; en caso contrario, muestre “La matriz no es cuadrada”. (En este ejemplo debe mostrar que no es cuadrada).

Estructura ifelse

`ifelse()` es una función vectorizada que permite evaluar una condición para cada elemento de un vector y devolver un resultado según la condición sea verdadera o falsa.

Puede aplicarse a vectores, matrices y columnas de un data frame.

No controla el flujo general del programa como `if...else`; genera resultados para cada elemento.

Ejemplo 9

Considere el vector `edad <- c(15, 18, 20, 16, 35, 14, 20)`. Utilice la función `ifelse()` para clasificar cada edad como “Mayor” si es mayor o igual a 18 años, o como “Menor” en caso contrario.

Ejemplo 10

Considere una matriz de tres columnas construida con los valores 1, 5, 7, 2, 3, 5, 2, 5 y 5. Utilice la función `ifelse()` para reemplazar cada elemento por el texto “Menor a 4” cuando su valor sea menor o igual a 3, y por “Mayor a 3” cuando sea mayor que 3.

Ejemplo 11

Considere el siguiente data frame que contiene el código y la nota de cinco estudiantes. Cree una nueva columna llamada Calificación, asignando el valor “Aprobado” a los estudiantes con nota mayor o igual a 10.5 y “Desaprobado” a los demás estudiantes.

```
notas <- data.frame(Alumno = c("001","002","008","015","025"),  
                    Nota    = c(12,10,15,18,19))
```

Estructuras repetitivas

Las estructuras repetitivas permiten ejecutar un conjunto de instrucciones varias veces. `for` y `while` son las principales en R.

`for`

La estructura `for` permite repetir un bloque de código recorriendo, uno por uno, los elementos de un objeto. En cada iteración, la variable toma uno de los valores de la secuencia.

Ejemplo 12

Muestre en pantalla los números enteros del 1 al 5, uno por cada iteración.

Ejemplo 13

Considere el vector `x <- c(5, 8, 2, 7)`. Recorra cada elemento del vector y mostrar en pantalla el resultado de sumar 10 a cada uno de ellos.

Ejemplo 14

Considere el vector $x \leftarrow 1:5$. Calcule la expresión $x^2 + 2x$ para cada elemento del vector y almacene los resultados en un nuevo vector llamado

Ejemplo 15

Considere el siguiente vector $x \leftarrow c(-5, 8, -2, 7, 0, 12)$. Crear un nuevo vector llamado resultado, reemplazando los valores negativos por 0 y conservando sin cambios los valores no negativos.

Ejemplo 16

Considere el siguiente data frame que contiene el código y la nota de cinco estudiantes.

```
notas <- data.frame(Alumno = c("001","002","008","015","025"),  
                    Nota    = c(12,10,15,18,19))
```

Cuente cuántos estudiantes obtuvieron una nota mayor o igual a 10.5, sin usar la función `count`. Guarde el resultado en un objeto llamado `contador`.

Ejemplo 17

Considere el siguiente vector `x <- c(14, 8, 23, -2, 17, 5, 20)`. Determine el valor máximo del vector y la posición en la que se encuentra, sin utilizar la función `max()`.

while

La estructura while permite repetir un bloque de código mientras una condición sea verdadera. A diferencia de for, el número de iteraciones no se conoce de antemano. El ciclo o bucle finaliza cuando la condición deja de cumplirse.

Dentro del ciclo debe existir alguna instrucción que modifique la condición, de no ser así, el ciclo podría ejecutarse indefinidamente.

Ejemplo 18

Inicialice un objeto llamado `i` con el valor 1. Utilice un bucle `while` para mostrar en pantalla los números enteros del 1 al 5.

Ejemplo 19

Inicialice un objeto llamado `x` con el valor 1. Utilice un ciclo `while` para mostrar sucesivamente los valores obtenidos al multiplicar `x` por 3, siempre que su valor sea menor o igual a 200. Además, en cada iteración, muestre el mensaje: “El valor de `x` es ...”

Ejemplo 20

Utilice el método de Newton para aproximar la raíz cuadrada de 16. Considere los siguientes valores iniciales:

```
numero      <- 16  
aproximacion <- 1  
error       <- 1  
tolerancia  <- 0.0001
```

Mediante un ciclo `while`, calcule nuevas aproximaciones utilizando la expresión:

$$aprox_i = \frac{aprox_{i-1} + numero/aprox_{i-1}}{2}$$

El ciclo debe continuar mientras el error sea mayor que la tolerancia establecida.

Estructuras de control de ciclos

Las estructuras de control de ciclos permiten modificar el comportamiento normal de una estructura repetitiva. Se utilizan para finalizar anticipadamente un ciclo o para omitir determinadas iteraciones.

`break`

La instrucción `break` permite finalizar inmediatamente un ciclo, aunque la condición de repetición todavía se cumpla o aún queden elementos por recorrer. Dicho de otro modo, cuando R encuentra `break`, abandona completamente el ciclo y continúa ejecutando las instrucciones que se encuentran después de este.

Ejemplo 21

Utilice un ciclo `for` para recorrer los números enteros del 1 al 10. Emplee la instrucción `break` para finalizar el ciclo cuando el contador sea igual a 6. Muestre en pantalla únicamente los valores recorridos antes de que el ciclo termine.

Ejemplo 22

Considere el siguiente vector `x <- c(4, 8, 12, 8, 20, 25)`. Utilice un ciclo `for` para recorrer el vector e imprimir el índice y el valor de cada elemento visitado. Cuando encuentre la primera aparición del valor 8, almacene su posición en un objeto llamado `posicion`, muestre un mensaje indicando la posición encontrada y finalice el ciclo.

next

La instrucción `next` permite omitir la iteración actual de un ciclo y continuar directamente con la siguiente. A diferencia de `break`, `next` no finaliza completamente el ciclo. Solo evita que se ejecuten las instrucciones restantes de la iteración que se está ejecutando.

Ejemplo 23

Recorra los números enteros del 1 a 10, omitiendo los múltiplos de 3 y muestre en pantalla únicamente los valores no omitidos.

Ejemplo 24

Considere el siguiente vector `x <- c(9, -4, 16, -1, 25, 0)`. Calcule la raíz cuadrada de cada elemento no negativo y almacene los resultados en un vector llamado `raices`. No se debe intentar calcular la raíz cuadrada de los valores negativos.

Funciones

Una función es un bloque de código diseñado para realizar una tarea en particular. Puede recibir datos de entrada, procesarlos y devolver un resultado.

Evita repetir código, permite organizar mejor programas extensos y son la base para la automatización de procesos y creación de paquetes.

Su estructura general es:

```
nombre_funcion <- function(argumentos){  
  instrucciones  
  resultado  
}
```

Ejemplo 25

Cree una función llamada `primera` que no reciba argumentos y muestre en pantalla el mensaje: "Esta es mi primera función". Luego, ejecute la función.

Ejemplo 26

Cree una función llamada `area_circulo` que reciba como argumento el radio de un círculo y devuelva su área utilizando la fórmula $A = \pi r^2$. Utilice la función para calcular el área de un círculo de radio 5.

Ejemplo 27

Cree una función llamada `potencia` que reciba un número x y un exponente p , cuyo valor por defecto sea 2. La función debe devolver x^p .

Ejemplo 28

Cree una función llamada `resumir_vector` que reciba un vector numérico `x` y un argumento opcional `remove_na`, cuyo valor por defecto sea `TRUE`.

La función debe realizar las siguientes tareas:

1. Verificar que el objeto recibido sea un vector numérico. En caso contrario, mostrar un mensaje de error.
2. Verificar que todos los valores se encuentren entre 0 y 20. Si existe algún valor fuera de ese rango, detener la ejecución.
3. Calcular y devolver un vector con las siguientes medidas descriptivas: mínimo, media, mediana, máximo.

Finalmente, utilice la función para resumir el siguiente vector de notas

```
notas <- c(1,8,9,6,15,14,11,NA,12)
```

Ejercicio integrador

Una empresa desea automatizar el control de inventario de sus productos. Se dispone del siguiente data frame:

```
productos <- data.frame(  
  Codigo    = c("P001", "P002", "P003", "P004", "P005", "P006", "P007", "P008"),  
  Precio    = c(25, 18, NA, 45, -10, 80, 1500, 32),  
  Cantidad  = c(15, 8, 20, 0, 10, 5, 30, 12))
```

Un producto será considerado válido únicamente cuando: - El precio no sea NA. - El precio sea mayor que 0. - El precio sea menor o igual a 1000. - La cantidad sea mayor que 0.

Cree una función llamada `procesar_productos()` que realice las siguientes tareas:

1. Verifique que el objeto recibido sea un `data.frame`.
2. Verifique que existan las columnas `Precio` y `Cantidad`.
3. Recorra las filas mediante un ciclo `for`.
4. Utilice `next` para omitir los productos que no cumplan las condiciones anteriores.
5. Clasifique cada producto válido como:
 - ▶ “Económico”, si su precio es menor a S/ 50.
 - ▶ “Costoso”, en caso contrario.
6. Utilice `break` para detener el procesamiento cuando se hayan evaluado cinco productos válidos.

7. Devuelva una lista que contenga:

- ▶ el inventario procesado
- ▶ el número de productos económicos
- ▶ el número de productos costosos
- ▶ el precio promedio
- ▶ el producto más caro
- ▶ el producto más barato